Nonlinear Inverse Problems & Stochastic Inversion

Jochen Kamm & Pankaj K Mishra

April 20, 2026

Learning goals

By the end of this lecture, the aim is to understand:

- recap the main ideas from the previous lecture: forward modelling, inverse problems, objective functions, and regularization,
- connect those ideas to hands-on deterministic inversion with Newton and Gauss–Newton,
- understand the difference between local search and global search in non-linear inverse problems,
- see why stochastic inversion is a useful global option when the objective surface is complicated,
- learn the basic idea of PSO, inspired by flocking birds and collective motion,
- learn the basic idea of annealing-based search, including VFSA, inspired by how heated metals are cooled toward low-energy states,
- understand why uncertainty matters and how model ensembles help us think beyond one single best model.

Intuitive goal

The main objective is to move from a recap of local inversion toward global search, stochastic exploration, and uncertainty in inverse problems.

What is a non-linear inverse problem? (Forward model)

Forward problem: predict data from a model $\mathbf{m}$.

Linear forward model

$$\mathbf{d}^{\text{pre}} = \mathbf{G}\,\mathbf{m}, \qquad \mathbf{G} \in \mathbb{R}^{N \times M}, \qquad \mathbf{d}^{\text{obs}} = \mathbf{G}\,\mathbf{m} + \varepsilon.$$

Non-linear forward model

$$\mathbf{d}^{\text{pre}} = \mathbf{g}\left(\mathbf{m}\right), \qquad \mathbf{d}^{\text{obs}} = \mathbf{g}\left(\mathbf{m}\right) + \varepsilon.$$

$$\mathbf{g}\left(\alpha \mathbf{m}_1 + \beta \mathbf{m}_2\right) \neq \alpha\,\mathbf{g}\left(\mathbf{m}_1\right) + \beta\,\mathbf{g}\left(\mathbf{m}_2\right).$$

What is a non-linear inverse problem? (Inverse task)

Inverse problem: given observations $\mathbf{d}^{\text{obs}}$, estimate $\mathbf{m}$.

$$\mathbf{m}^{\text{est}} = \arg\min_{\mathbf{m}} \Phi(\mathbf{m})$$

A common objective function

$$\Phi(\mathbf{m}) = \underbrace{\left(\mathbf{d}^{\text{obs}} - \mathbf{g}(\mathbf{m})\right)^T \mathbf{C}_{\mathbf{d}}^{-1} \left(\mathbf{d}^{\text{obs}} - \mathbf{g}(\mathbf{m})\right)}_{\textit{data misfit}} + \epsilon^2 \underbrace{L(\mathbf{m})}_{\textit{regularization / prior}} .$$

- $\mathbf{g}(\mathbf{m})$ is non-linear $\Rightarrow$ the minimizer is usually not available in closed form.
- We therefore solve the inverse problem iteratively (linearize $\mathbf{g}(\mathbf{m})$ locally).

Why do we need iterative methods?

Near a current model $\mathbf{m}_i$, we locally linearize the forward operator:

$$\mathbf{g}(\mathbf{m}_i + \Delta\mathbf{m}) \approx \mathbf{g}(\mathbf{m}_i) + \mathbf{G}_i \Delta\mathbf{m}, \qquad \mathbf{G}_i = \left.\frac{\partial \mathbf{g}(\mathbf{m})}{\partial \mathbf{m}}\right|_{\mathbf{m}=\mathbf{m}_i}.$$

- A non-linear problem is difficult globally, but often manageable locally.
- So we solve a sequence of local approximations.
- At each iteration we:
  - linearize around the current model,
  - compute an update $\Delta\mathbf{m}$,
  - and move to $\mathbf{m}_{i+1} = \mathbf{m}_i + \Delta\mathbf{m}$.
- This is why the result can depend strongly on the starting model.

Newton's method: the main idea

Newton's method approximates the objective function locally by a quadratic:

$$\Phi(\mathbf{m}_i + \Delta\mathbf{m}) \approx \Phi(\mathbf{m}_i) + \nabla\Phi(\mathbf{m}_i)^T\Delta\mathbf{m} + \frac{1}{2}\Delta\mathbf{m}^T\mathbf{H}(\mathbf{m}_i)\Delta\mathbf{m}.$$

The update is then

$$\Delta\mathbf{m} = -\mathbf{H}(\mathbf{m}_i)^{-1}\nabla\Phi(\mathbf{m}_i), \qquad \mathbf{m}_{i+1} = \mathbf{m}_i + \Delta\mathbf{m}.$$

- Newton's method uses the local slope and curvature of the objective.
- Intuitively: if the objective looked quadratic near the current point, where would its minimum be?
- That gives the next model, and the process is repeated.

Gradient and Hessian: what do they mean?

The two key ingredients in Newton's method are

$$\nabla\Phi(\mathbf{m}) \qquad \text{and} \qquad \mathbf{H}(\mathbf{m}).$$

$$\nabla\Phi(\mathbf{m}) = \text{gradient}, \qquad \mathbf{H}(\mathbf{m}) = \text{Hessian}.$$

- The gradient tells us the local **direction of change** of the objective.
- The Hessian tells us the local **curvature** or shape of the objective surface.
- In Newton's method:
  - the gradient tells us *which way to move*,
  - the Hessian tells us *how the surface bends*, so it scales the step.
- This makes Newton more informative than simple gradient descent.

Other local optimization methods

Many local optimizers use the same objective function, but differ in how much derivative information they use.

Common examples in machine learning

- **Gradient descent**: uses only $\nabla\Phi(\mathbf{m})$ and moves in the steepest downhill direction.
- **Adam**: also uses gradients, but adapts the step size using running averages of past gradients.
- **L-BFGS**: uses gradients and builds a low-memory approximation to curvature, without forming the full Hessian.

Relation to Newton and Gauss–Newton

These are still local optimizers. Newton and Gauss–Newton sit in the same family, but they use curvature information more directly.

- First-order methods: gradient descent, Adam.
- Quasi-Newton methods: L-BFGS.
- Second-order methods: Newton, or approximate second-order methods such as Gauss–Newton.

Newton method: high-level algorithm

$$\mathbf{H}(\mathbf{m}_i)\Delta\mathbf{m} = -\nabla\Phi(\mathbf{m}_i), \qquad \mathbf{m}_{i+1} = \mathbf{m}_i + \Delta\mathbf{m}.$$

- Choose a starting model $\mathbf{m}_0$.
- Evaluate the forward response $\mathbf{g}(\mathbf{m}_i)$.
- Compute the gradient $\nabla\Phi(\mathbf{m}_i)$ and Hessian $\mathbf{H}(\mathbf{m}_i)$.
- Solve for the update $\Delta\mathbf{m}$.
- Update the model and repeat until convergence.

- Newton can converge very fast near a good solution.
- But computing the full Hessian is often costly.

Gauss–Newton method: A practical version of Newton Method

Gauss–Newton keeps the Newton idea, but approximates the Hessian by dropping expensive second-derivative terms.

Here $\mathbf{f}(\mathbf{m})$ denotes the model-side penalty or regularization map, and $\mathbf{F}_i$ is its Jacobian evaluated at $\mathbf{m}_i$.

A common approximation is

$$\tilde{\mathbf{H}}(\mathbf{m}_i) \approx 2\left(\mathbf{G}_i^T \mathbf{C}_\mathbf{d}^{-1} \mathbf{G}_i + \epsilon^2 \mathbf{F}_i^T \mathbf{F}_i\right).$$

The update becomes

$$\Delta\mathbf{m} = \tilde{\mathbf{H}}(\mathbf{m}_i)^{-1}\left[\mathbf{G}_i^T \mathbf{C}_\mathbf{d}^{-1}\left(\mathbf{d} - \mathbf{g}(\mathbf{m}_i)\right) + \epsilon^2 \mathbf{F}_i^T\left(\mathbf{f}(\langle\mathbf{m}\rangle) - \mathbf{f}(\mathbf{m}_i)\right)\right].$$

- So Gauss–Newton is an approximation of Newton's method.
- $\mathbf{f}(\mathbf{m})$ measures how the current model is penalized relative to the prior or regularization target.
- $\mathbf{F}_i = \partial\mathbf{f}(/)\,\partial\mathbf{m}$ tells us how that penalty changes when the model changes.
- It is cheaper and easier to use in practice.
- It works well for many moderately non-linear problems.
- This is why Gauss–Newton is widely used.

Simplified covariance model in the scripts

The general notation uses covariance matrices $\mathbf{C_d}$ and $\mathbf{C_{m,prior}}$, while the scripts use a deliberately simpler special case.

Classroom simplification

$$\mathbf{C_d} = \sigma_d^2 \mathbf{I}, \qquad \mathbf{C_{m,prior}} = \mathbf{I}.$$

So the data misfit in the scripts is

$$\Phi_d(\mathbf{m}) = \frac{1}{2}\chi_d^2(\mathbf{m}) = \frac{1}{2\sigma_d^2}\left\|\mathbf{d}^{\text{obs}} - \mathbf{g}(\mathbf{m})\right\|_2^2.$$

- This keeps the example easy to read and easy to code.
- The main concepts are the same: weighted data fit, regularization, and iterative or stochastic search.
- Later, the same ideas extend to more realistic covariance models.

Code demo: deterministic inversion examples

The lecture demos for the deterministic part of this lecture are in the repository

`https://github.com/pankajkmishra/GEOM2057`

Main files

- `L6/01-problem-setup-derivatives-etc.jl`: forward model, objective, derivatives.
- `L6/02-data-generation.jl`: synthetic data generation.
- `L6/03-inversion-NewtonMethod.jl`: Newton inversion demo.
- `L6/04-inversion-GaussNewton.jl`: Gauss–Newton inversion demo.

Demo focus

The deterministic scripts connect the mathematics to the synthetic data, the iteration history, and the final recovered model.

Two practical ways to compute derivatives

There are several practical ways to obtain

$$\mathbf{G} = \frac{\partial \mathbf{g}(\mathbf{m})}{\partial \mathbf{m}}$$

are:

Finite differences

- Easy to understand and implement.
- Useful for checking code.
- But usually expensive.

Analytical derivatives

- If the forward map is simple enough, the derivatives can also be calculated analytically.
- This is often cleaner and more accurate than finite differences.

Adjoint method

- Very useful in large PDE-based inverse problems.
- More efficient when gradients are needed many times.

What changes in the stochastic examples?

We keep the same toy inverse problem, the same forward map, and the same objective structure.

What stays the same

- the forward model $\mathbf{g}(\mathbf{m})$,
- the three-parameter model vector $\mathbf{m} = [m_1, m_2, m_3]$,
- the weighted objective $\Phi(\mathbf{m}) = \Phi_d(\mathbf{m}) + \epsilon^2 \Phi_m(\mathbf{m})$.

What changes

- fewer data are used,
- the noise level is higher,
- we switch from local derivative-based updates to broader global-search strategies.

Why local methods may not be enough

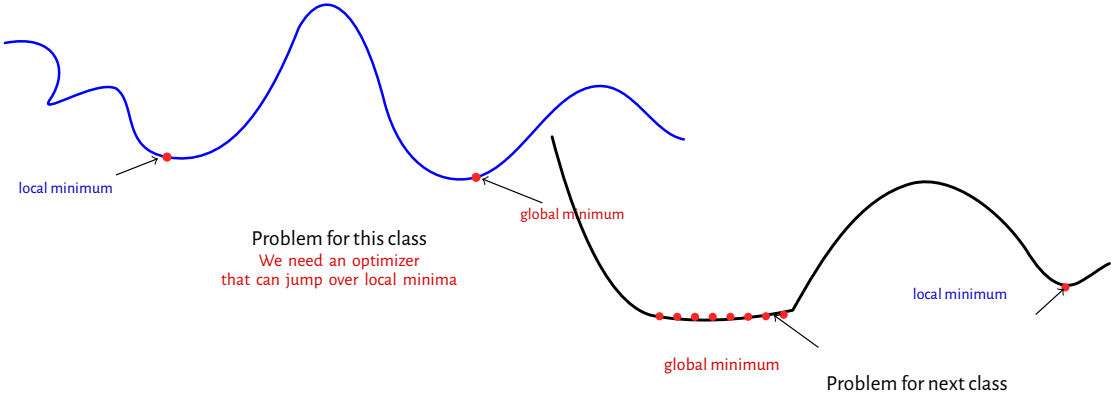
Local methods such as Newton and Gauss–Newton use nearby slope and curvature information.

- If the objective surface is smooth and bowl-shaped near the starting model, local methods can be excellent.
- But if the objective surface has several valleys, plateaus, or narrow basins, the final answer may depend strongly on the starting model.
- In our stochastic version of the problem, fewer data and larger noise make the objective surface less informative and potentially more multi-modal.

Motivation for the next methods

We now want methods that explore the search space more broadly, rather than following only local derivatives from one starting point.

Local minimum versus global minimum



Grid search: the simplest global search

Intuition: cover the search domain by a finite grid and test every candidate model.

Suppose each parameter m_j is restricted to an interval

$$m_j \in [m_j^{\min}, m_j^{\max}], \qquad j = 1, \ldots, M.$$

Choose grid values $m_j^{(1)}, \ldots, m_j^{(K_j)}$ and define

$$\mathcal{G} = \left\{ \mathbf{m} = (m_1^{(k_1)}, \ldots, m_M^{(k_M)}) : k_j \in \{1, \ldots, K_j\} \right\}.$$

Then we compute the objective on every grid point:

$$\mathbf{m}^{\text{est}} = \arg\min_{\mathbf{m} \in \mathcal{G}} \Phi(\mathbf{m}), \qquad \Phi(\mathbf{m}) = \Phi_d(\mathbf{m}) + \epsilon^2 \Phi_m(\mathbf{m}).$$

- This is the most direct way to search globally.
- It is extremely useful for teaching because the idea is transparent.
- It is also useful for visualizing 1D, 2D, or 3D objective surfaces.

Grid search: algorithm and limitation

Algorithm

- Choose lower and upper bounds for every parameter.
- Discretize each parameter interval.
- Evaluate $\Phi(\mathbf{m})$ for all $\mathbf{m} \in \mathcal{G}$.
- Keep the model with the smallest value.

Main limitation

The cost grows as $|\mathcal{G}| = \prod_{j=1}^{M} K_j$, so grid search is excellent for teaching and low-dimensional problems, but not for large parameter spaces.

- This rapid growth with dimension is often called the curse of dimensionality.
- Methods such as PSO and VFSA try to search globally without evaluating every possible grid point.

Families of global optimisation methods

Population-based methods

- PSO
- Genetic algorithms
- Ant colony optimization
- Many others inspired by natural processes

These methods evolve a population of candidate models.

Annealing-based methods

- Simulated annealing (SA)
- Very Fast Simulated Annealing (VFSA)

These methods follow a temperature-controlled stochastic search.

Position in this lecture

SA accepts occasional uphill moves with a probability controlled by temperature. VFSA is a faster, more efficient variant of SA that still allows large exploratory jumps early in the search.

Particle Swarm Optimisation

PSO: intuition and natural inspiration

Particle Swarm Optimisation (PSO) is inspired by collective motion in nature, such as

- bird flocks,
- fish schools,
- and other groups that move by combining individual behavior with group information.

Intuition

Each particle is a candidate model. A particle keeps moving through model space by combining three tendencies:

- keep some of its current motion,
- return toward the best location it has found itself,
- move toward the best location found by the swarm.

- The method is therefore both social and stochastic.
- It does not require derivatives of the objective function.

PSO: mathematical objects

For particle $p = 1, \ldots, N_p$ at iteration k:

State variables

$$\mathbf{m}_p^{(k)} : \text{particle position}, \qquad \mathbf{v}_p^{(k)} : \text{particle velocity}.$$

$$\mathbf{p}_p^{(k)} = \arg \min_{0 \le s \le k} \Phi\left(\mathbf{m}_p^{(s)}\right) : \text{personal best},$$

$$\mathbf{m}_g^{(k)} = \arg \min_{p,s} \Phi\left(\mathbf{m}_p^{(s)}\right) : \text{global best}.$$

Meaning of the terms

- ω: inertia weight
- c_1: pull toward personal best
- c_2: pull toward global best
- $r_1, r_2 \sim U(0,1)^M$: random exploration
- σ_m: swarm spread

Idea

PSO balances momentum, individual memory, and social information to move a swarm toward promising regions without using derivatives.

PSO: mathematical formulation

Main PSO update

$$\mathbf{v}_p^{(k+1)} = \omega\, \mathbf{v}_p^{(k)} + c_1\, r_1^{(k)} \odot \left(\mathbf{p}_p^{(k)} - \mathbf{m}_p^{(k)}\right) + c_2\, r_2^{(k)} \odot \left(\mathbf{m}_g^{(k)} - \mathbf{m}_p^{(k)}\right),$$

$$\mathbf{m}_p^{(k+1)} = \Pi_{\mathcal{M}} \left(\mathbf{m}_p^{(k)} + \mathbf{v}_p^{(k+1)}\right).$$

Here $\odot$ denotes componentwise multiplication and $\Pi_{\mathcal{M}}$ denotes projection back to the admissible model domain.

- The first term keeps momentum from the previous iteration.
- The second term pulls the particle toward its own personal best.
- The third term pulls the particle toward the swarm best.

PSO: algorithmic steps

Algorithm

- Initialize $\mathbf{m}_p^{(0)}$ and $\mathbf{v}_p^{(0)}$ inside the bounds.
- Evaluate $\Phi(\mathbf{m}_p^{(0)})$.
- Set personal bests $\mathbf{p}_p^{(0)}$ and swarm best $\mathbf{m}_g^{(0)}$.
- Update particles by the equations above.
- Replace personal and global bests whenever an improved objective is found.
- Stop after the prescribed number of iterations.

Key property

PSO is a derivative-free global optimiser. Exploration comes from the random vectors r_1, r_2 and from the fact that many particles search simultaneously.

What objective do PSO and VFSA minimize?

PSO and VFSA use the same objective as the deterministic examples; only the search strategy changes. If the focus is only on data fit, we usually do not need to write the full $\Phi(\mathbf{m})$ explicitly.

$$\Phi(\mathbf{m}) = \Phi_d(\mathbf{m}) + \epsilon^2 \Phi_m(\mathbf{m})$$

Weighted data misfit

$$\chi_d^2(\mathbf{m}) = \left(\mathbf{d}^{\text{obs}} - \mathbf{g}(\mathbf{m})\right)^T \mathbf{C}_\mathbf{d}^{-1} \left(\mathbf{d}^{\text{obs}} - \mathbf{g}(\mathbf{m})\right).$$

In the classroom scripts, $\mathbf{C}_\mathbf{d} = \sigma_d^2 \mathbf{I}$, so

$$\chi_d^2(\mathbf{m}) = \frac{\left\|\mathbf{d}^{\text{obs}} - \mathbf{g}(\mathbf{m})\right\|_2^2}{\sigma_d^2},$$

and therefore

$$\Phi_d(\mathbf{m}) = \frac{1}{2}\chi_d^2(\mathbf{m}).$$

- So the global-search methods still minimize a noise-weighted fit to the data.
- RMS can be reported as a diagnostic, but the search itself is based on the weighted objective.

Code demo: PSO example

The PSO demo is in the repository

`https://github.com/pankajkmishra/GEOM2057`

Main files

- `L6/05-problem-setup-hard.jl`: shared hard-problem setup for the comparison case.
- `L6/06-data-generation-hard.jl`: shared noisy data generation for the comparison case.
- `L6/07-inversion-GaussNewton-hard.jl`: local Gauss–Newton comparison on the hard case.
- `L6/08-inversion-PSO-hard.jl`: difficult non-linear PSO demo.

Demo focus

The PSO demo illustrates swarm exploration, the difference between current particle quality and global best, and the contraction of the swarm toward good regions.

Simulated annealing: intuition and natural inspiration

Simulated annealing (SA) is inspired by thermal annealing in materials science.

- A material is heated so atoms can move freely.
- It is then cooled gradually so the system settles into a low-energy state.

Optimisation analogy

Replace physical energy by the objective function $\Phi(\mathbf{m})$ and replace temperature by a control parameter T.

- High temperature allows broad exploration.
- Low temperature encourages settling near good minima.
- Unlike deterministic descent, SA can accept some uphill moves.

Very Fast Simulated Annealing

VFSA: proposal mechanism

Very Fast Simulated Annealing (VFSA) is a special variant of simulated annealing designed to keep the same exploration philosophy while using a faster cooling strategy and efficient long-range perturbations.

VFSA builds a perturbation variable y_j for each parameter direction and uses it to generate the next trial model.

Heavy-tailed random perturbation

Choose a random number

$$u_j \sim U(0,1),$$

and transform it into a Cauchy-type perturbation

$$y_j = \operatorname{sgn}\left(u_j - \frac{1}{2}\right) T \left[\left(1 + \frac{1}{T}\right)^{|2u_j - 1|} - 1\right].$$

Intuition

The variable y_j is heavy-tailed, so most jumps are moderate but occasional large jumps are still possible. This is what helps VFSA escape narrow local minima.

VFSA: from perturbation to trial model

Once y_j has been generated, the new model is formed componentwise by

VFSA proposal mechanism

$$m_{\mathrm{prop},j} = \Pi_{[m_j^{\min}, m_j^{\max}]}\left(m_{\mathrm{current},j} + y_j\left(m_j^{\max} - m_j^{\min}\right)\right),$$

Here $\Pi_{[m_j^{\min}, m_j^{\max}]}$ enforces the parameter bounds.

- $\mathbf{m}_{\mathrm{current}}$: current model in the random walk.
- u_j: uniform random number.
- y_j: heavy-tailed perturbation in the jth parameter.
- $m_j^{\min}, m_j^{\max}$: lower and upper bounds for parameter j.

Intuition

The perturbation y_j first decides *how far to move*. The factor $(m_j^{\max} - m_j^{\min})$ scales that jump to the size of the admissible parameter range, and the projection operator keeps the proposal inside the allowed bounds.

VFSA: acceptance rule

After proposing a new model, VFSA compares the objective values through

$$\Delta\Phi = \Phi(\mathbf{m}_{\mathrm{prop}}) - \Phi(\mathbf{m}_{\mathrm{current}}).$$

Acceptance rule

$$P_{\mathrm{acc}} = \begin{cases} 1, & \Delta\Phi \leq 0, \\ \exp\left(-\dfrac{\Delta\Phi}{T}\right), & \Delta\Phi > 0. \end{cases}$$

- If $\Delta\Phi \leq 0$, the new model improves the objective and is accepted.
- If $\Delta\Phi > 0$, the new model is worse, but it may still be accepted.

Role of temperature in the acceptance step

When T is high, uphill moves are accepted more easily, so the search can escape local minima. When T is low, uphill moves are rarely accepted, so the search focuses on refinement near good solutions.

VFSA: cooling schedule

The temperature is reduced gradually according to a cooling law.

Cooling schedule

$$T_k = \max\left(\frac{T_0}{1 + 0.15(k-1)}, 10^{-3}\right).$$

- At the beginning, T_k is large, so proposals are bold and uphill moves are still possible.
- As k increases, T_k decreases, so both the perturbations and the acceptance of worse moves become smaller.
- The search therefore evolves from broad exploration to local refinement.

Intuition

Temperature is the mechanism that controls the transition from searching widely to settling carefully.

VFSA: algorithmic steps

Algorithm

- Choose an initial model and initial temperature T_0.
- For each temperature level, generate several trial moves.
- Compute $\Delta\Phi$ and accept with probability P_{acc}.
- Update the best-so-far model whenever improvement occurs.
- Lower the temperature and continue.

Key property

VFSA is a single-chain derivative-free global search method. It explores by random proposals and temperature-controlled acceptance rather than by gradients or swarm motion.

Code demo: VFSA example

The VFSA demo is in the repository

`https://github.com/pankajkmishra/GEOM2057`

Main files

- `L6/05-problem-setup-hard.jl`: shared hard-problem setup for the comparison case.
- `L6/06-data-generation-hard.jl`: shared noisy data generation for the comparison case.
- `L6/09-inversion-VFSA-hard.jl`: difficult non-linear VFSA demo.

Demo focus

The VFSA demo illustrates temperature, random proposals, occasional uphill acceptance, and the difference between broad early exploration and later refinement.

Why uncertainty matters in geophysical inversion

In geophysical inversion, a single best-fit model is rarely the whole story.

Main idea

Different models can explain the observed data almost equally well, especially when the data are noisy, sparse, or insensitive to some parameters.

- We want not only an estimate $\mathbf{m}^{\text{est}}$, but also an idea of how certain or how uncertain that estimate is.
- Uncertainty tells us which features of the model are strongly constrained and which are ambiguous.
- This matters for interpretation, decision making, and planning further data acquisition.

What creates uncertainty?

Data-side uncertainty

- measurement noise,
- incomplete spatial coverage,
- limited resolution,
- inconsistent or biased observations.

Model-side uncertainty

- non-uniqueness,
- regularization choices,
- parameterization choices,
- imperfect forward physics.

Consequence

Even if we find a model with small $\Phi(\mathbf{m})$, there may still be a broad family of acceptable models around it.

How do we describe uncertainty in this lecture?

In this lecture, we use a practical viewpoint: uncertainty means that there is not just one acceptable model, but a family of acceptable models.

- Some model parameters may be tightly constrained, while others may vary a lot with little change in data fit.
- We can therefore talk about model spread, acceptable ranges, and families of plausible solutions.
- This gives a useful uncertainty picture even before we introduce formal sampling theory.

Practical summary tools

We summarize uncertainty through model ensembles, spreads, variability of parameters, and sensitivity of the solution to starting models and noise.

Why do we rarely do this fully in 3D?

Core difficulty

In 3D inversion, the number of unknowns is often extremely large, so uncertainty analysis becomes much more expensive than finding one best-fit model.

- A 3D model may contain thousands to millions of parameters.
- Each forward simulation can already be expensive.
- Stochastic uncertainty analysis requires many forward evaluations, not just one inversion run.
- Storage and analysis of many 3D models can also become a serious computational burden.

What is a model ensemble?

A model ensemble is a collection of acceptable models rather than a single estimate:

Ensemble

$$\mathcal{E} = \left\{ \mathbf{m}^{(1)}, \mathbf{m}^{(2)}, \ldots, \mathbf{m}^{(N)} \right\}.$$

- Each member of the ensemble is a plausible model according to the data and the inversion assumptions.
- The spread of the ensemble gives a practical picture of uncertainty.
- Ensemble statistics can summarize central tendency and variability.

Simple ensemble summaries

$$\bar{\mathbf{m}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{m}^{(i)}, \qquad \sigma_{m,j}^2 = \frac{1}{N} \sum_{i=1}^{N} \left(m_j^{(i)} - \bar{m}_j \right)^2.$$

How can VFSA produce a model ensemble?

One practical approach is to run multiple independent VFSA chains.

Basic idea

- Start VFSA from many different initial models.
- Run each chain with independent random perturbations.
- Keep either the final accepted models or a set of low-objective models from each chain.
- Merge these models into one ensemble.

Ensemble from multiple chains

$$\mathcal{E} = \bigcup_{r=1}^{R} \left\{ \mathbf{m}_r^{(k)} : k \in \mathcal{K}_r \right\},$$

where r indexes the chain and $\mathcal{K}_r$ denotes the saved states from chain r.

What do we learn from such an ensemble?

- If all chains converge to a narrow cluster of similar models, uncertainty is relatively small.
- If different chains produce very different but still acceptable models, uncertainty is large and non-uniqueness is important.
- We can inspect marginal spreads, correlations between parameters, and robust versus unstable model features.

Important caution

An ensemble built from VFSA runs is a practical uncertainty tool, but its quality still depends on proposal design, cooling schedule, run length, and selection criteria.

Code demo: ensemble idea from multiple VFSA chains

The uncertainty extension can also be built from the same repository

`https://github.com/pankajkmishra/GEOM2057`

Demo focus

- Several VFSA runs can start from different initial models.
- Final or low-objective models from each run can be collected.
- The spread of accepted models across runs can then be compared.

Main message

This turns the optimisation demo into a first uncertainty demo: not one answer, but a family of plausible models with visible spread.

Bridge to the next lecture: MCMC

Today we used global search and ensembles to build intuition about uncertainty. In the next lecture, we will make this more formal using MCMC methods.

- VFSA gives us a practical way to explore multiple acceptable models.
- MCMC will take the next step and treat uncertainty through a more systematic probabilistic sampling framework.
- So the current lecture builds the intuition, and the next lecture provides the formal machinery.

So what did we learn?

- A non-linear inverse problem has a forward operator $\mathbf{d} = \mathbf{g}(\mathbf{m})$ that is not globally linear.
- Because of that, we usually solve it iteratively.
- Newton's method uses gradient and Hessian to minimize a local quadratic approximation.
- Gauss–Newton makes Newton practical by approximating the Hessian.
- Deterministic inversion is usually fast and informative, but it can depend strongly on the starting model and may miss distant good solutions.
- Stochastic methods such as PSO and VFSA explore more broadly and can reveal multiple acceptable models.
- They are powerful for complex objective landscapes, but they are also more computationally expensive.
- So deterministic and stochastic inversion each have their own advantages and limitations.

After this lecture, you should be able to explain the difference between local and global search, describe the ideas behind PSO and VFSA, and interpret uncertainty in terms of families of acceptable models rather than one single answer.